

Reflux Sentinel — Complete Project Description

1. What It Is

Reflux Sentinel is a wearable GERD (gastroesophageal reflux disease) monitoring device designed for people managing chronic acid reflux. It clips to the waist, tracks posture, sleep position, meals, symptoms, and heart rate variability, and delivers real-time audio and haptic feedback when it detects risky behaviour — such as lying down too soon after eating, or sleeping on the wrong side.

The device is fully offline and air-gapped. It never connects to the internet. Communication with the user's phone happens exclusively over a local WiFi hotspot the device broadcasts. All data stays on-device in a SQLite database, synced to the phone only when the user explicitly connects.

The total parts budget for the prototype is approximately 550–600 Lei.

2. Design Philosophy

Every decision flows from a few core principles:

Compact and discreet. The device must be comfortable for all-day wear on a belt or waistband, hard to notice under clothing, and light enough that the user forgets it's there within 20 minutes. Target weight is under 60g for the clip unit alone.

Fully offline. No cloud, no internet, no Telegram, no BLE — ever. The Pi broadcasts its own WiFi hotspot. The device never joins any external network. This was a deliberate architectural decision, not a limitation. It eliminates dependency on external services, simplifies the software stack, and keeps health data entirely under the user's control.

No internal battery. Power comes from the user's existing 10,000 mAh power bank in a pocket, connected via a short USB-C cable with an XT30 inline quick-swap connector. This

keeps the clip unit thin and light, avoids LiPo BMS complexity, and gives effectively unlimited runtime.

No mic, no LED. Audio is output-only (speaker + vibration motor). No always-listening microphone, no status LEDs. The OLED display on the back face handles all visual feedback.

Event-driven, not polling. The software architecture reacts to state changes rather than continuously sampling. This keeps CPU usage minimal, thermals low, and power draw manageable on the Pi Zero 2 W's single-core ARM.

3. Hardware Specification (Locked)

3.1 Platform

Raspberry Pi Zero 2 W — quad-core ARM Cortex-A53, 512MB RAM, built-in WiFi (used for hotspot), built-in Bluetooth (unused — disabled). Runs Raspberry Pi OS Lite. All development in Python.

3.2 Form Factor

Horizontal slab, approximately **80mm × 40mm × 20mm**. Comparable in size to a thick pager or a glucose monitor. Matte finish PETG enclosure with 3–4mm radius on all corners and edges for comfort. Monolithic shell — no visible screws on front or sides.

Front face (faces outward when worn):

- OV9712 ultrawide USB UVC camera at one end — for food photo capture
- Speaker grilles at the opposite end — patterned holes for the 28–32mm speaker

Top edge:

- Tactile button on the left — primary action/back button
- EC11 rotary encoder with push-click on the right — scroll/select/confirm ("crown"); 10nF capacitors for hardware debounce

Back face (faces the body, visible through clip window):

- SSD1306 1.3" or SH1106 OLED display, centred — shows status, menus, QR codes, food results, onboarding
- Display and encoder/button are accessible through an open window in the clip

Clip mechanism:

- Lamy-style PETG blade that curves over the back of the device body
- An open window in the clip exposes the OLED display and controls
- The clip grips belt/waistband fabric between the blade and the device body
- PETG chosen for flexibility and fatigue resistance over PLA

Inner face of clip (against skin):

- MAX30102 optical heart rate sensor — pressed against skin by the clip's tension
- Small foam pad behind the MAX30102 for consistent contact pressure
- Signal quality indicator read programmatically; HRV pauses automatically when no skin contact detected

3.3 Power System

- **Source:** User's existing 10,000 mAh power bank, kept in a pocket
- **Connection:** Short USB-C cable from power bank to the device's USB-C port on the short edge
- **Quick-swap:** XT30 inline connector on the cable — robust click-lock, allows instant battery swap without shutting down (capacitor hold-up on the Pi side)
- **No internal battery, no LiPo, no BMS**

3.4 Sensors & Output

Component	Model	Purpose	Interface	Location
IMU	MPU6050 GY-521	Posture (pitch/roll), gyro activity, sleep position	I2C (GPIO 2/3)	Inside enclosure
HRV sensor	MAX30102	Heart rate variability, stress correlation	I2C	Clip inner face
Audio amp	MAX98357A	Drives speaker for voice alerts	I2S	Inside enclosure
Speaker	28–32mm 8Ω 1W	Voice output (pre-generated WAV files)	Wired to amp	Front face
Vibration	Coin type	Haptic nudges (silent alerts,	GPIO +	Inside

motor		sleep)	transistor	enclosure
Display	SSD1306 1.3" / SH1106	Status, menus, food results, QR code	I2C	Back face
Camera	OV9712 USB UVC	Food photo capture for TFLite analysis	USB OTG	Front face
Action button	Tactile switch	Primary action, back, meal/symptom logging	GPIO interrupt	Top left
Rotary encoder	EC11 with click	Scroll, select, confirm	GPIO (3 pins)	Top right

Wiring approach: Direct point-to-point GPIO wiring. No breadboard. Headers desoldered from breakout boards to minimise height. Short wires routed through internal cable channels moulded into the PETG case walls.

4. Connectivity Architecture

4.1 WiFi Hotspot (Only Network Interface)

The Pi runs `hostapd` and broadcasts its own WiFi network. The device **never joins any external network**. When the user wants to sync data, view dashboards, or change settings, they connect their phone to the device's hotspot.

- **Max clients:** 1–2 simultaneous connections
- **Hotspot control:** User sets on/off schedule hours via the phone PWA, or toggles manually from the device's OLED/encoder
- **QR code:** The OLED can display a QR code encoding the hotspot SSID and password — one scan on iOS 13+ or Android 10+ auto-joins the network, no typing
- **Access point IP:** 192.168.1.1 — the Flask PWA lives here

4.2 What Was Removed (and Why)

- **BLE** — removed entirely. Adds protocol complexity, library dependencies (bleak), and pairing headaches. WiFi hotspot handles everything BLE would do, with higher bandwidth and zero pairing friction.
- **Tailscale** — removed. Required internet access, which contradicts the air-gapped

design.

- **Telegram bot** — removed. Required internet access. Morning reports and notifications now appear on the Flask dashboard when the phone connects.
 - **Cloud backend** — removed from prototype. Reserved for Stage 2/3 scaling.
-

5. Software Architecture

5.1 Core Principles

- **Event-driven:** Modules communicate through an internal event bus (Python queue). No module calls another directly. `mode_manager.py` is the sole consumer that orchestrates state transitions.
- **systemd-managed:** Main process runs as a systemd service with `Restart=on-failure`. Automatic recovery from crashes.
- **Hot-reloadable config:** `config.json` watched via Python `watchdog` library. Changes from the phone PWA take effect immediately without restart.
- **CPU governor:** `powersave` by default, temporarily bumped during TFLite inference only.

5.2 Module Structure

```
reflux_sentinel/
├─ main.py           # Entry point, signal handling, event bus setup, systemd
├─ config.py         # Loads/watches config.json, provides typed access to al
├─ imu.py            # MPU6050 – pitch/roll/gyro reads, posture angle, activi
├─ hrv.py            # MAX30102 – HRV calculation, signal quality check, skin
├─ alerts.py         # WAV playback (MAX98357A), vibration motor patterns, co
├─ logger.py         # SQLite wrapper – WAL mode, batch writes every 30s, sch
├─ camera.py         # OV9712 capture, powers camera on/off, feeds frames to
├─ button.py         # GPIO interrupt handler for tactile button – press patt
├─ encoder.py        # EC11 rotary encoder – rotation events, click events, d
├─ foods.py          # TFLite MobileNetV2 inference, food risk dictionary (10
├─ sleep_tracker.py  # Sleep position classification, nudge scheduling, morni
├─ mode_manager.py   # Central FSM – orchestrates all state transitions, cons
├─ hotspot.py        # hostapd management – bring up/tear down, monitor conne
├─ meds.py           # Medication reminder timers – fires alerts, tracks ackn
├─ web/
│   └─ app.py        # Flask PWA server – all routes, API endpoints, WebSocke
│   └─ templates/    # Jinja2 templates for dashboard, settings, reports, onb
└─ audio/           # Pre-generated WAV files via Piper TTS – all voice aler
```

5.3 Database (SQLite with WAL)

Tables:

- `sessions` — device power-on/off events, wear-side, calibration baseline
- `events` — meals, symptoms (severity 1–3), food photos, medication acknowledgments
- `posture_log` — timestamped pitch/roll/activity, sparse (logged on state change, not continuously)
- `sleep_log` — sleep position samples, nudge events, morning summary data

Write strategy: Batch writes every 30 seconds. State-change logging only — not continuous time-series. This keeps I/O manageable on the Pi Zero's SD card.

Calibration persistence: Personal calibration baselines stored in SQLite (not just `config.json`), surviving reboots and config resets.

5.4 Sensor Sampling Rates

- **Baseline:** 2 Hz (MPU6050 pitch/roll)
- **Post-meal window (2–3 hours):** 5 Hz
- **Sleep mode:** 0.03 Hz (one sample every ~33 seconds)
- **Camera:** Fully powered off until triggered by user action; draws 0mA when off

6. Finite State Machine (`mode_manager.py`)

The FSM has **8 states** with explicit transitions:

6.1 States

State	Description
BOOTING	Hardware init, sensor checks, config load, display splash
ONBOARDING	First-boot flow — language, wear side, calibration, sleep window
IDLE	Normal monitoring — posture tracking at 2Hz, alerts active
	Triggered by meal log — elevated sampling (5Hz), stricter posture

POST_MEAL	thresholds, meal countdown timer active
FOOD_PHOTO	Camera active, TFLite inference running, result shown on OLED, awaiting crown click confirmation
PRE_SLEEP	Transitional — pitch flat detected, 10-minute stillness countdown before entering full sleep
SLEEPING	Ultra-low-power monitoring — 0.03Hz sampling, vibration-only nudges, position classification
CALIBRATING	User-initiated recalibration of posture baseline via OLED + encoder

6.2 Key Transition Rules

- **POST_MEAL blocks sleep entry** — cannot transition to PRE_SLEEP while meal countdown is active
- **FOOD_PHOTO is gated** — only accessible from POST_MEAL or IDLE, not during sleep states
- **PRE_SLEEP requires all three conditions:** pitch flat + 10 minutes of stillness + time within sleep window (default 11pm–7am, configurable)
- **Medication reminders queue** during FOOD_PHOTO and CALIBRATING states rather than interrupting
- **Lying-down grace period:** 2 minutes of escalating haptic warnings before voice alert fires (prevents false alarms from briefly sitting down)

6.3 Edge Cases Sealed

- Sleep cannot activate mid-meal (POST_MEAL blocks it)
 - Camera cannot fire in wrong state (gated by FSM)
 - Config changes during TFLite inference are deferred
 - Double food photo triggers prevented (camera state locked during inference)
 - Power bank disconnect recovery (capacitor hold-up, graceful shutdown write)
 - Hotspot dying mid-sync (auto-retry with backoff)
-

7. Features (Complete List)

7.1 Posture Monitoring

- Pitch angle from MPU6050 with rolling average smoothing
- Gyro-based activity detection (walking vs. sitting vs. lying)
- Personal calibration — user-specific baseline angles
- Wear-side-aware (left hip vs. right hip) — configured during onboarding, changeable in settings
- Contextual thresholds — stricter limits during post-meal window

7.2 Meal & Symptom Logging

- **Tactile button press patterns:** single press = mild symptom (severity 1), double = moderate (2), triple = severe (3)
- **Long press** = meal event logged
- Each event timestamped and stored in SQLite

7.3 Food Photo Analysis

- User presses button to trigger camera
- OV9712 captures frame → TFLite MobileNetV2 runs inference on-device
- Result matched against a food risk dictionary (100+ entries with GERD risk scores)
- Result displayed on OLED (food name, risk level)
- User confirms with crown click → SQLite write + spoken advice plays (8–15 second pre-generated WAV)
- For ambiguous results (confidence <60%): "I'm not sure what that is — log it manually"
- Camera fully powers off after capture

7.4 HRV & Stress Monitoring

- MAX30102 reads PPG signal when skin contact detected
- HRV calculated and logged alongside posture and meal events
- Stress/HRV trends correlated with meals in the Flask dashboard — novel clinical insight

- Signal quality indicator: automatically pauses HRV when no skin contact, logs gap

7.5 Alerts System

- **Voice alerts:** Pre-generated WAV files via Piper TTS, played through MAX98357A + speaker
- **Haptic alerts:** Coin vibration motor patterns — gentle nudge, moderate pulse, strong buzz
- **Escalation:** Haptic → voice, with configurable cooldown periods between alerts
- **Lying-down grace:** 2-minute escalating haptic warnings before voice alert fires post-meal
- **Sleep-specific:** Vibration only, max 3 escalating pulses per night, no voice

7.6 Hydration Reminders

- Time-based reminders when no drink has been logged within a configurable window
- Simple audio/haptic nudge

7.7 Meal Countdown

- After meal logging, a 2–3 hour (configurable) countdown timer tracks the upright window
- Stricter posture monitoring during this period
- Countdown visible on OLED and Flask dashboard

7.8 Medication Reminders

- Configured via phone PWA during first-time setup or anytime after
- Timed reminders (e.g., “Take PPI 30 minutes before breakfast”)
- Acknowledged via top tactile button click on the device
- Editable and viewable via phone
- Queued during sensitive states (food photo, calibration) — delivered after

7.9 Sleep Monitoring

- **Auto-detection:** Pitch flat + 10 minutes of stillness + time within sleep window
- **Position classification:** MPU6050 roll angle determines left side (good for GERD), right

side, back, or front (all suboptimal). Wear-side-aware — the algorithm adjusts based on which hip the device is clipped to.

- **Sampling:** Drops to 0.03 Hz in sleep mode
- **Nudges:** Vibration only, max 3 escalating pulses per night. No voice to avoid waking partner.
- **Morning report:** Generated and stored; appears on Flask dashboard when phone next connects

7.10 OLED Display & Controls

- **OLED (back face, through clip window):** Status info, menu navigation, food photo results, QR code for hotspot, onboarding flow, calibration interface
- **Tactile button (top left):** Action / back / meal-symptom logging via press patterns
- **Rotary encoder (top right):** Scroll through menus, adjust settings, click to confirm. Functions as the “crown” — inspired by smartwatch interaction

7.11 First-Boot Onboarding

- OLED-guided flow on first power-up
- Steps: language selection → wear side (left/right hip) → posture calibration → sleep window hours → medication setup prompt
- Avoids requiring phone connection for initial configuration

7.12 Trigger Confirmation Loop

- After each alert, user can crown-click to confirm “yes, I felt symptoms”
- This feedback is stored and used to personalise the food risk model over time — the device learns which foods actually trigger the individual user, rather than relying purely on generic risk tables

8. Web Application (Flask PWA)

8.1 Access Model

- Served at **192.168.1.1** over the device’s WiFi hotspot
- Phone connects to hotspot → opens browser → PWA loads

- First visit detects iOS/Android and version → shows platform-specific “Add to Home Screen” instructions
- No internet required. No external access. Purely local.

8.2 Routes & Features

Dashboard:

- Live posture angle (WebSocket or polling from Unix socket)
- Timeline of today’s events (meals, symptoms, alerts, medication)
- Quick-log buttons (meal, symptom, water)

Settings:

- Wear side selection
- Posture threshold sliders
- Sleep window hours
- Nudge configuration (frequency, max per night, escalation timing)
- Food risk table (view and edit individual food scores)
- Remote calibration trigger
- Hotspot schedule (on/off hours or manual-only)
- Medication reminder setup
- Auto-backup toggle

Sleep Chart:

- Night-by-night position breakdown (% left, right, back, front)
- Nudge log
- Sleep score

Food Log:

- Photo thumbnails with identified food, risk score, and user confirmation
- Personal trigger map (foods correlated with confirmed symptoms)

Reports:

- CSV/JSON export of all data
- Phone generates PDF locally from JSON (no server-side PDF generation)
- Doctor-friendly summary format

8.3 Auto-Backup

- Triggered automatically each time a phone connects to the hotspot (toggleable)
 - SQLite database + config.json backed up to the phone via HTTP download
-

9. Audio System

- **Piper TTS** used offline to pre-generate all WAV files at build time
 - WAV files stored in `audio/` directory — one file per alert phrase
 - Playback via `aplay` or Python `wave` + `pyaudio` through the MAX98357A I2S amp
 - Typical alert duration: 8–15 seconds
 - No runtime TTS — instant playback, zero CPU overhead at alert time
 - Potential future improvement: custom-recorded calm clinical voice rather than default Piper voice
-

10. Physical Design Details

10.1 Enclosure

- Material: PETG (preferred over PLA for flexibility, fatigue resistance, and matte finish)
- Matte finish — either naturally from PETG print settings or light sand + primer
- Soft edges: 3–4mm radius on all corners
- No visible screws on front or sides — recessed screws on back panel only
- Internal cable channels moulded into walls for wire routing
- Optional two-tone: main body one colour, clip blade in contrasting darker colour

10.2 Weight & Comfort

- Target: under 60g for clip unit alone (power bank separate, in pocket)
- Pi Zero 2 W centred inside for weight distribution — prevents torque and digging
- Worn at left or right hip, slightly forward of hip bone — natural body hollow
- Hidden under untucked shirt or light jacket
- Cable exits bottom edge, drops straight into pocket — coiled cable or short right-angle USB-C minimises slack

10.3 Aesthetic Target

From a metre away: looks like a pager, a small radio, or a medical device. From up close: intentional, matte, medical-grade feel. Not hobbyist.

11. Optional: Night Pod

A separate wrist-worn unit for sleep-specific monitoring, independent of the main waist device:

- **MCU:** ESP32-C3
- **Sensor:** MPU6050 (sleep position via roll angle)
- **Output:** Coin vibration motor (wrist-based haptic nudges)
- **Form factor:** Wrist strap
- **Communication:** WiFi to the main Pi device, or standalone with its own local storage
- **Use case:** Users who don't want to sleep with the waist clip on

This is scoped but not yet implemented. Could potentially be a standalone simpler product for sleep-only GERD monitoring.

12. Scaling Roadmap

Stage 1 — Prototype (Current)

- Pi Zero 2 W, breakout boards, 3D-printed PETG enclosure
- Power bank in pocket
- Direct GPIO wiring, no PCB

- Python + Flask + SQLite
- Cost per unit: ~550–600 Lei in parts
- Single user (David)

Stage 2 — Small Batch Pilot (10–50 units)

- **Platform:** CM4 (Compute Module 4) or ESP32-S3
- **Battery:** Integrated flat LiPo cell (1000–2000 mAh) with BMS
- **PCB:** Custom board designed in KiCad, manufactured via JLCPCB (\$15–30 for 10 boards)
- **Camera:** CSI ribbon module (replacing USB UVC)
- **Enclosure:** SLA 3D print (smoother finish)
- **App:** Flutter companion app (cross-platform, one codebase)
- **Software:** OTA update support, systemd-managed
- **Backend:** Cloud sync optional — FastAPI + PostgreSQL
- **Cost per unit:** ~80–150 Lei at this volume

Stage 3 — Product (100+ units)

- **Regulatory:** EU MDR 2017/745 compliance. Posture monitor = likely Class I (self-declare). Anything claiming to predict/prevent reflux = Class IIa (needs Notified Body + clinical validation).
- **Hardware:** Full custom PCBA from contract manufacturer, pre-certified BLE module (Nordic/u-blox for CE marking), IP54-rated enclosure, medical-grade skin-contact materials
- **Backend:** Full cloud infrastructure with clinician dashboard, HL7 FHIR export for hospital system integration
- **Cost per unit:** ~40–80 Lei at volume

Pilot Path

- **Target:** UMF Craiova gastroenterology department
- **Format:** Research study under ethics committee review (not a commercial device launch — avoids MDR certification requirement at this stage)

- **Scale:** 10–20 patients
 - **Goal:** Real-world data, potential academic paper, proof of concept for funding
-

13. Business Model & Open Source Strategy

Pricing (by stage)

- **Prototype/pilot:** Free, open source, no subscriptions. Priority is adoption, feedback, clinical credibility.
- **Stage 2:** Hardware sale (~400–700 Lei) + optional cloud Pro tier (~40–60 Lei/month or ~400 Lei/year). Free tier covers local-only use (Flask dashboard, local logs). Pro unlocks cloud sync, doctor-shareable reports, long-term trend analysis.
- **Stage 3:** B2B2C model — gastroenterology departments or insurers pay per-patient for monitoring programme. Patient gets device free or at nominal cost.

Open Source Approach

- **Recommended:** Open source the firmware and phone app; keep cloud backend and clinical data infrastructure proprietary
 - **Rationale:** Builds trust with patients and clinicians (they can audit exactly what handles their health data), aligns with EU GDPR transparency requirements, fits Hack Club and Horizon Europe open science requirements, attracts contributors
 - **Hardware design files:** Open under non-commercial license — allows research use, prevents direct commercial copying
 - **Proprietary moat:** Food risk model tuning, personalised trigger data, clinical validation work — these are the real competitive advantages, not the firmware
-

14. Funding Paths Being Tracked

Source	Type	Fit
Hack Club (Highway / Grants)	Student/maker grants	Strong — David has existing Hack Club familiarity
	EU health innovation	Strong — specifically targets early-stage

EIT Health	programmes	health tech
Horizon Europe	EU SME/startup instruments	Good — has open science requirements that align with open source approach
Romanian PNRR	Digital health component of national recovery funds	Good — Romanian institutions can apply

15. Project Evolution Timeline

The project evolved across **4 conversations** in this Claude project:

1. **Conversation 1 (April 3):** Initial concept brought from a prior conversation. Defined the core hardware (Pi Zero 2 W, MPU6050, MAX98357A, OV9712), software architecture (event-driven Python, SQLite WAL, Flask, systemd), connectivity (originally Tailscale + Telegram + BLE), form factor (initially vertical slab, then horizontal), scaling stages, pilot path (UMF Craiova), and funding landscape. Confirmed no breadboard needed — direct GPIO wiring. Explored clip mechanism, HRV sensor placement (clip inner face against skin), battery options (power bank in pocket vs. magnetic snap-on vs. integrated LiPo), connectors (USB-C external, XT30 for power, JST-SH for internal I2C). Camera and speaker options evaluated. Memories created.
2. **Conversation 2 (April 4, early):** Feature expansion and cost optimisation. Added: lying-down grace period with escalating haptics, HRV/stress correlation in dashboard, medication reminders, trigger confirmation loop (double-tap trains personal food risk model), calibration persisted to SQLite, second tactile button, OLED onboarding flow, mode_manager as proper FSM. Removed ambient context inference. Discussed cost reduction options (drop camera saves ~100 Lei, smaller OLED saves ~15–20 Lei, ESP32-S3 saves ~60–100 Lei). Explored pricing models and open source strategy. **Major architecture change:** removed BLE, Tailscale, Telegram, and all internet connectivity entirely. Replaced with WiFi hotspot-only model. Added hotspot scheduling, QR code for auto-join, auto-backup on connection.
3. **Conversation 3 (April 4, mid):** Improvement review. Identified stale references in memories (Tailscale, Telegram, BLE still mentioned). Added foam pad behind MAX30102 for consistent pressure. Confirmed QR code on OLED for WiFi join. Split `sensor.py` into `imu.py` and `hrv.py`. Added `encoder.py`, `hotspot.py`, and `meds.py` as distinct modules. Locked the connectivity model: Pi hotspot only, max 1–2 clients, scheduled or manual. Confirmed morning report is pull-based (Flask dashboard). Locked food photo flow: TFLite → OLED result → crown click confirms → SQLite + spoken advice. Removed ambient context inference. Added first-visit PWA install prompt with platform-specific

instructions.

4. **Conversation 4 (April 4, latest):** FSM architecture deep-dive. Defined 8 FSM states (BOOTING, ONBOARDING, IDLE, POST_MEAL, FOOD_PHOTO, PRE_SLEEP, SLEEPING, CALIBRATING) with explicit transition triggers and guard conditions. Sealed all identified edge cases. Defined the event bus pattern (modules produce events, mode_manager consumes). Established implementation build order for Claude Code.
-

16. David's Working Style & Preferences

- Prefers **ideas, outlines, and architectural thinking** over direct code at the planning stage
 - Works iteratively: locks hardware spec first → software architecture → implementation details
 - Thinks in scaling stages from the start (prototype → pilot → product)
 - Considers regulatory and funding landscape early, even at prototype phase
 - Has FTC robotics background — comfortable with CAD, hardware, and electromechanical systems
 - Strong with Python, Flask, Linux, Pi ecosystem, Tailscale, systemd
 - Plans to use **Claude Code** for implementation — paste project summary at session start for instant context recovery
 - Based in Craiova, Romania
-

17. Key Technical Learnings Accumulated

- Pre-generated WAV files via Piper TTS give near-instant audio playback without runtime TTS overhead
- SQLite WAL with sparse state-change logging (not continuous) keeps storage and I/O manageable on Pi Zero's SD card
- Camera must stay fully powered off until triggered — critical for thermal and power management
- Breadboard is unnecessary and counterproductive — direct GPIO wiring keeps form factor achievable

- WiFi hotspot sidesteps all connectivity complexity (BLE pairing, internet dependency, cloud ops)
 - EC11 rotary encoder needs hardware debounce (10nF capacitors) — software debounce alone is unreliable
 - MAX30102 on clip inner face needs foam pad behind sensor for consistent skin pressure
 - Event bus pattern (Python queue) decouples all modules — enables testing on MacBook without any hardware present
 - `config.json` hot-reload via `watchdog` lets phone settings changes take effect instantly
 - Hotspot with QR code on OLED eliminates the most painful UX step (typing WiFi password on phone)
-

18. Open Questions & Future Decisions

- **Companion app framework:** Flutter currently favoured for Stage 2 (cross-platform from single codebase). React Native was considered but Flutter has better embedded/IoT ecosystem.
 - **Night pod integration scope:** Could be standalone product or tightly coupled. Standalone is simpler for regulatory and manufacturing. Decision deferred.
 - **Custom Piper TTS voice:** Default voice works but a calm, clinical custom voice would improve pilot study perception. Low priority for now.
 - **FHIR export format:** Needed for Stage 3 hospital integration but irrelevant until then.
 - **IP rating for pilot units:** IP54 minimum desired but challenging with 3D-printed PETG at small scale. May need silicone gaskets.
-

This document represents the complete, current state of the Reflux Sentinel project as of April 4, 2026. All earlier specifications that conflict with this document (e.g., references to BLE, Tailscale, Telegram, cloud connectivity, `ble_server.py`, `sensor.py` as a single module) are superseded.